

การใช้งาน SQL Server

ทั้งในมุมมอง System Admin และ Developer แบบครบวงจร

ครอบคลุมตั้งแต่การติดตั้ง ดูแลระบบ ความปลอดภัย ประสิทธิภาพ
ไปจนถึงการออกแบบฐานข้อมูล การเขียน T-SQL และ Best Practices จริง

26 SLIDES

T-SQL EXAMPLES

DIAGRAMS

WORKSHOP

1. SQL Server คืออะไร

Microsoft SQL Server — RDBMS ระดับองค์กร

► ความหมาย

- ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (RDBMS) ของ Microsoft
- จัดเก็บ ดึง และประมวลผลข้อมูลผ่าน T-SQL
- รองรับทั้ง OLTP, OLAP และ Big Data

► จุดเด่น

- ความน่าเชื่อถือสูง มี ACID transactions
- Security ระดับ enterprise (TDE, Always Encrypted)
- รองรับ Windows / Linux / Container / Azure

► Editions หลัก

- Enterprise – เพิ่มความสามารถ สำหรับองค์กรขนาดใหญ่
- Standard – ครบสำหรับธุรกิจทั่วไป
- Developer – ฟรีสำหรับพัฒนา/ทดสอบ เทียบเท่า Enterprise
- Express – ฟรีสำหรับงานขนาดเล็ก (10 GB/DB)

► Ecosystem

- SSMS – เครื่องมือบริหารจัดการหลัก
- Azure Data Studio – cross-platform IDE
- SSIS / SSRS / SSAS – ETL, รายงาน, วิเคราะห์
- Power BI และ Azure SQL เชื่อมต่อได้ไร้รอยต่อ

2. SQL Server ใช้ในงานประเภทใด

ครอบคลุมตั้งแต่ระบบหลังบ้านจนถึง Analytics



ERP / CRM

SAP, Dynamics และระบบองค์กร



Banking & Finance

ระบบธุรกรรมความถูกต้องสูง



E-Commerce

สั่งซื้อ สต็อก ชำระเงิน



Healthcare

เวชระเบียน ข้อมูลผู้ป่วย



Government

ทะเบียนประชาชน ภาษี



Data Warehouse

BI, Power BI, Reporting

แนวโน้ม: การย้ายขึ้น Cloud (Azure SQL) และ Hybrid เพิ่มมากขึ้นในองค์กรทั่วโลก

3. บทบาทของ Admin และ Developer

ขอบเขตงานที่แตกต่างแต่ต้องทำงานร่วมกัน

SYSTEM ADMIN / DBA

- ▶ ติดตั้งและกำหนดค่า SQL Server
- ▶ จัดการ Instance, Database, File
- ▶ ดูแล Security, Login, Permission
- ▶ Backup / Restore / DR Plan
- ▶ Monitoring, Maintenance, Patching
- ▶ Performance tuning ระดับ Server
- ▶ High Availability / Cluster

DEVELOPER

- ▶ ออกแบบ schema / ER Diagram
- ▶ เขียน T-SQL / CRUD Queries
- ▶ สร้าง Stored Procedure / Function / View
- ▶ จัดการ Transaction & Concurrency
- ▶ ออกแบบ Index & ทำ Query Tuning
- ▶ Error Handling & ป้องกัน SQL Injection
- ▶ เชื่อมต่อฐานข้อมูลกับ Application

4. SQL Server Architecture

ภาพรวมสถาปัตยกรรมและ Engine หลัก

SQL SERVER INSTANCE



Protocol Layer · TDS / TCP / Named Pipes / Shared Memory

Relational Engine

- ▶ Command Parser
- ▶ Query Optimizer
- ▶ Query Executor

Storage Engine

- ▶ Access Methods
- ▶ Buffer Manager
- ▶ Transaction Manager & Lock Manager



Storage · Data Files (.mdf, .ndf) · Log File (.ldf) · TempDB · Backup Files (.bak)

5. การติดตั้ง SQL Server และ SSMS

ขั้นตอนการติดตั้งระบบพร้อมใช้งาน

01

ดาวน์โหลด Installer

เลือก Edition ที่เหมาะสม (Dev/Express สำหรับเรียน)

02

Installation Type

New installation หรือ Add features

03

Feature Selection

Database Engine, Full-Text, Integration Services

04

Instance Config

Default หรือ Named Instance เลือก path ข้อมูล

05

Authentication

Windows / Mixed Mode กำหนดรหัส sa

06

ติดตั้ง SSMS

ดาวน์โหลดแยก — เครื่องมือหลักในการบริหาร

💡 Tip: ใช้ Developer Edition สำหรับสภาพแวดล้อมเรียน/ทดสอบ — ฟรี และความสามารถเท่า Enterprise

6. Instance, Database, File และ TempDB

โครงสร้างจัดเก็บข้อมูลของ SQL Server

SQL SERVER INSTANCE

master

model

msdb

tempdb

System Databases

Sales_DB

HR_DB

Shop_DB

User Databases

-  **.mdf** Primary data file
-  **.ndf** Secondary data file(s)
-  **.ldf** Transaction log file

TempDB — ทำไมต้องสนใจ?

- ▶ DB ชั่วคราวที่ทุก user ใช้ร่วมกัน
- ▶ เก็บ temp tables, table variables, sort/hash workspaces
- ▶ รีเซ็ตทุกครั้งที่เรา restart SQL Server
- ▶ ควรแยก disk และมีหลาย data files (เท่า logical CPUs, สูงสุด 8)
- ▶ ติดตาม contention ที่ PAGELATCH_* บน allocation pages
- ▶ กำหนดขนาดเริ่มต้นและ auto-growth ให้เท่ากัน

7. Security และ Permission

หลักการรักษาความปลอดภัยแบบหลายชั้น

01

Authentication

ใครเข้ามาได้บ้าง? (Login)

02

Authorization

เข้าแล้วทำอะไรได้? (User/Role/Permission)

03

Encryption

ข้อมูลถูกเข้ารหัส (TDE, Always Encrypted, TLS)

04

Auditing

ใครทำอะไรเมื่อไหร่ (SQL Audit, Extended Events)

Permission Hierarchy

Server — sysadmin, securityadmin

Database — db_owner, db_datareader

Schema — dbo, sales, hr

Object — Table, View, SP

Column — sensitive columns

8. Login / User / Role

จัดการสิทธิ์อย่างเป็นระบบด้วย Role

► Login

ระดับ Server — ใช้ login เข้า SQL Server instance

► User

ระดับ Database — map จาก login เพื่อเข้าถึง DB

► Role

กลุ่มสิทธิ์ที่กำหนดล่วงหน้า (Fixed/Custom)

●●● T-SQL · Login / User / Role

```
-- สร้าง Login ระดับ Server
CREATE LOGIN dev_user WITH PASSWORD = 'StrongP@ss2026!'

-- สร้าง User ใน Database และ map กับ Login
USE Shop_DB
CREATE USER dev_user FOR LOGIN dev_user

-- ให้สิทธิ์ผ่าน Role (แนะนำ)
ALTER ROLE db_datareader ADD MEMBER dev_user
GRANT EXECUTE ON SCHEMA::sales TO dev_user
```

9. Backup / Restore / Recovery Model

หัวใจของการปกป้องข้อมูล

SIMPLE

Log ตัดเอง เสีย log ช่วง backup ล่าสุด

FULL

เก็บ log ครบ กู้ point-in-time ได้

BULK - LOGGED

Log น้อยลงสำหรับ bulk ops

Backup Strategy — ตัวอย่าง 3-2-1 Chain

FULL

Weekly · Sun

DIFF

Daily · Mon–Sat

LOG

Every 15 min

OFFSITE

Encrypted Copy

●●● T-SQL · Backup

```
BACKUP DATABASE Shop_DB TO DISK = 'D:\Backup\Shop_FULL.bak' WITH COMPRESSION, CHECKSUM
BACKUP LOG Shop_DB TO DISK = 'D:\Backup\Shop_LOG.trn' WITH COMPRESSION
```

10. Maintenance และ Monitoring

งานประจำที่ต้องทำสม่ำเสมอ

Update Statistics

ช่วยให้ optimizer เลือ ก plan ดีขึ้น

Weekly

Rebuild/Reorganize Index

ลด fragmentation เกิน 30%

Weekly

DBCC CHECKDB

ตรวจสอบ corruption ของ database

Weekly

Backup Verify

RESTORE VERIFYONLY / test restore

Monthly

Monitor CPU/Memory/IO

Perf Counter, DMVs, Query Store

Continuous

Alerts & Notifications

DB Mail + SQL Agent alerts

Continuous

💡 ใช้ Query Store เพื่อดูประวัติ query plans และหา regression อัตโนมัติ

11. Performance Tuning

ขั้นตอนการ tune ระบบให้รีดประสิทธิภาพสูงสุด



● ● ● DMV · หา query ที่ใช้ CPU มากที่สุด

```
SELECT TOP 10 * FROM sys.dm_exec_query_stats ORDER BY total_worker_time DESC
```

12. High Availability และ Disaster Recovery

ทำให้ระบบไม่ล่มและกู้คืนได้เร็ว

Always On AG

Availability Groups — ขั้นสูงที่สุด รองรับ read-only replicas



FCI

Failover Cluster Instance — shared storage, auto-failover



Log Shipping

ส่ง log ไป secondary ตั้งเวลาสลับ failover manual



Backup & Restore

พื้นฐานที่สุด — RTO/RPO ขึ้นกับความถี่ backup



13. Database Design

ออกแบบอย่างมีหลักก่อนเริ่มสร้าง table

Normalization (1NF → 3NF)

1NF ค่าต้องเป็น atomic ไม่ซ้ำในแถว

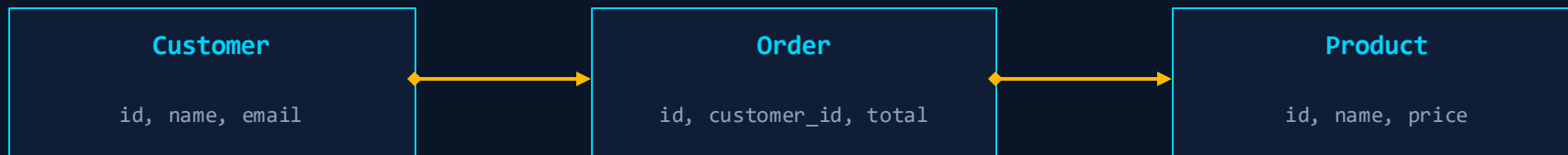
2NF ไม่มี partial dependency จาก composite key

3NF ไม่มี transitive dependency

Design Principles

- ▶ ตั้งชื่อสื่อความหมาย สอดคล้องทั้งระบบ
- ▶ ใช้ Surrogate Key (INT IDENTITY) เป็นหลัก
- ▶ แยก lookup tables สำหรับ enum/status
- ▶ กำหนด FK บังคับความสมบูรณ์ของข้อมูล
- ▶ Denormalize เฉพาะเมื่อจำเป็น (reporting)

ตัวอย่าง ER Diagram



14. Table, PK, FK และ Relation

สร้าง schema ที่มีความสมบูรณ์

T-SQL · Create Table with PK/FK

```
CREATE TABLE Customer (  
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,  
    FullName NVARCHAR(100) NOT NULL,  
    Email NVARCHAR(150) NOT NULL UNIQUE,  
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()  
)  
  
CREATE TABLE [Order] (  
    OrderID INT IDENTITY(1,1) PRIMARY KEY,  
    CustomerID INT NOT NULL,  
    OrderDate DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    TotalAmount DECIMAL(12,2) NOT NULL CHECK (TotalAmount >= 0),  
    CONSTRAINT FK_Order_Customer FOREIGN KEY (CustomerID)  
        REFERENCES Customer(CustomerID)  
)
```

15. Data Types

เลือก type ที่ถูกต้อง — ส่งผลทั้ง storage และ performance

หมวด	Type	ใช้เมื่อ
Integer	TINYINT / SMALLINT / INT / BIGINT	ตัวเลขไม่มีทศนิยม
Decimal	DECIMAL(p,s) / NUMERIC	เงิน ค่าที่ต้องแม่นยำ
Float	FLOAT / REAL	งานวิทยาศาสตร์ (approx)
String	CHAR / VARCHAR / NVARCHAR	N = Unicode (ภาษาไทย)
Date/Time	DATE / DATETIME2 / DATETIMEOFFSET	DATETIME2 แนะนำแทน DATETIME
Binary	VARBINARY(MAX) / FILESTREAM	ไฟล์ รูปภาพ
Others	BIT / UNIQUEIDENTIFIER / XML / JSON	BIT = boolean 0/1

💡 ภาษาไทยต้องใช้ NVARCHAR เสมอ · DATETIME2 มี precision สูงกว่า DATETIME เดิม

16. Basic T-SQL

โครงสร้างคำสั่งพื้นฐานของภาษา T-SQL

DDL

CREATE · ALTER · DROP ·
TRUNCATE

DML

SELECT · INSERT · UPDATE ·
DELETE · MERGE

DCL

GRANT · REVOKE · DENY

TCL

BEGIN TRAN · COMMIT ·
ROLLBACK · SAVE

●●● T-SQL · Basic SELECT

```
-- Variable และ SELECT
DECLARE @MinPrice DECIMAL(10,2) = 100

SELECT TOP 10
    p.ProductID, p.Name, p.Price
FROM Product AS p
WHERE p.Price >= @MinPrice AND p.IsActive = 1
ORDER BY p.Price DESC
```

17. CRUD Operations

Create · Read · Update · Delete

●●● T-SQL · CREATE / READ

```
-- CREATE
INSERT INTO Customer (FullName, Email)
VALUES ('สมชาย ใจดี', 'somchai@ex.com')

-- READ
SELECT CustomerID, FullName, Email
FROM Customer
WHERE Email LIKE '%@ex.com'
```

●●● T-SQL · UPDATE / DELETE

```
-- UPDATE
UPDATE Customer
SET FullName = N'สมชาย มีสุข'
WHERE CustomerID = 1

-- DELETE
DELETE FROM Customer
WHERE CustomerID = 99
```

18. JOIN และ Aggregate

รวมข้อมูลจากหลาย table และสรุปผล

INNER JOIN

แถวที่ match ทั้งสองฝั่ง

LEFT JOIN

ทุกแถวจากซ้าย + ที่ match ขวา

RIGHT JOIN

ทุกแถวจากขวา + ที่ match ซ้าย

FULL JOIN

ทุกแถวจากทั้งสองฝั่ง

T-SQL · JOIN + GROUP BY + HAVING

```
-- JOIN + Aggregate: ยอดซื้อรวมรายลูกค้า
SELECT
    c.CustomerID, c.FullName,
    COUNT(o.OrderID)      AS OrderCount,
    SUM(o.TotalAmount)    AS TotalSpent,
    AVG(o.TotalAmount)    AS AvgOrderValue
FROM Customer AS c
LEFT JOIN [Order] AS o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.FullName
HAVING SUM(o.TotalAmount) > 10000
ORDER BY TotalSpent DESC
```

19. Stored Procedure / View / Function

Reusable code objects ของฐานข้อมูล

Stored Procedure

โค้ดทำหลาย statement รับ param คืน result/output

View

query ที่ save ไว้ มองเป็น table ใช้ซ้ำและ abstract

Function

Scalar/Table-valued คืนค่าเรียกใน SELECT/WHERE

T-SQL · Stored Procedure

```
CREATE OR ALTER PROCEDURE usp_GetTopCustomers
    @TopN INT = 10
AS
BEGIN
    SET NOCOUNT ON
    SELECT TOP (@TopN) c.CustomerID, c.FullName,
        SUM(o.TotalAmount) AS Total
    FROM Customer c JOIN [Order] o ON o.CustomerID = c.CustomerID
    GROUP BY c.CustomerID, c.FullName
    ORDER BY Total DESC
END
GO
```

20. Transaction และ Concurrency

ACID และการจัดการเข้าพร้อมกัน

- A** Atomicity — ทั้งหมดหรือไม่ทำเลย
- C** Consistency — คงสถานะถูกต้อง
- I** Isolation — ต่างธุรกรรมไม่รบกวนกัน
- D** Durability — commit แล้วไม่หาย

Isolation Levels

- ▶ READ UNCOMMITTED (dirty read)
- ▶ READ COMMITTED (default)
- ▶ REPEATABLE READ
- ▶ SERIALIZABLE (เข้มงวดสุด)
- ▶ SNAPSHOT (แนะนำสำหรับ read-heavy)

●●● T-SQL · Simple Transaction

```
BEGIN TRAN
UPDATE Account SET Balance -= 500 WHERE AcctID = 1
UPDATE Account SET Balance += 500 WHERE AcctID = 2
IF @@ERROR <> 0 ROLLBACK ELSE COMMIT
```

21. Index และ Query Optimization

กุญแจของ performance ฝั่ง Developer

Clustered Index

เรียงข้อมูลจริงในตาราง — 1 ตัวต่อ table

Non-Clustered

โครงสร้างแยกชี้กลับไป row data — หลายตัวได้

Covering Index

ครอบคลุม column ที่ query ใช้ทั้งหมด

Filtered Index

เฉพาะ subset ของแถว (WHERE clause)

●●● T-SQL · Non-Clustered + INCLUDE + Filtered

```
CREATE NONCLUSTERED INDEX IX_Order_Customer_Date
ON [Order] (CustomerID, OrderDate DESC)
INCLUDE (TotalAmount)
WHERE Status = 'Completed' -- Filtered Index
```

22. Error Handling และ SQL Injection

เขียนโค้ดปลอดภัยและทนต่อ error

●●● TRY / CATCH

```
BEGIN TRY
    BEGIN TRAN
    UPDATE Account SET Bal -= 500
    WHERE AcctID = 1
    UPDATE Account SET Bal += 500
    WHERE AcctID = 2
    COMMIT
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK
    THROW -- ส่ง error ต่อ
END CATCH
```

✗ Vulnerable (String Concat)

```
sql = "SELECT * FROM User WHERE name='" + input + "'"
```

Input: 'OR 1=1 -- → เข้าได้ทุก user

✓ Safe (Parameterized Query)

- ▶ ใช้ sp_executesql หรือ prepared statement
- ▶ ตรวจสอบ input validation ทุกชั้น
- ▶ Least Privilege สำหรับ DB account ของ app
- ▶ ใช้ ORM/parameter binding จาก app layer

23. การเชื่อมต่อ SQL Server กับ Application

Driver และ Connection Pattern

.NET / C#

Microsoft.Data.SqlClient · Entity Framework Core ·
Dapper

Java

Microsoft JDBC Driver · JPA / Hibernate

Node.js

mssql package · Tedious · Prisma / TypeORM

Python

pyodbc · pymssql · SQLAlchemy

●●● Connection String

```
// Connection String (.NET example)
Server=tcp:myserver,1433Database=Shop_DB
User Id=app_userPassword=***Encrypt=True
TrustServerCertificate=FalseConnection Timeout=30
```


24. Best Practices สำหรับองค์กร

แนวทางปฏิบัติที่ดีทั้ง Admin และ Developer



Security

Least Privilege · MFA · TDE · audit logs



Backup

3-2-1 strategy · test restore ทุกเดือน



Monitoring

baseline · alert · Query Store เปิดเสมอ



Performance

review execution plan · index ตาม workload



Schema

naming convention · FK constraints · version control



Code Quality

parameterized · TRY/CATCH · code review



Documentation

ER diagrams · runbook · DR drill



Deployment

DB migration scripts · CI/CD pipeline

25. Workshop Exercises · 26. Key Takeaways

ลงมือทำและบทสรุปของหลักสูตร



WORKSHOP EXERCISES

- ▶ ออกแบบ schema สำหรับระบบร้านค้า 5 table
- ▶ สร้าง table พร้อม PK/FK และ constraints
- ▶ เขียน CRUD + JOIN หายอดขายรายลูกค้า
- ▶ สร้าง Stored Procedure โอนเงินระหว่างบัญชี
- ▶ Tune query ช้าด้วย index + execution plan
- ▶ สร้าง Login/User + กำหนด Role ให้ทีม dev
- ▶ ทดสอบ Full + Log backup แล้ว restore



KEY TAKEAWAYS

- ▶ Admin และ Developer ต้องทำงานร่วมกัน
- ▶ Security ทำหลายชั้นและใช้ Least Privilege
- ▶ Backup ไม่พอ — ต้องทดสอบ restore ด้วย
- ▶ วัด baseline ก่อน tune เสมอ
- ▶ Parameterize query ทุกครั้ง · ไม่ต่อ string
- ▶ Index ดีกว่าเยอะ — แต่ต้องดูแลสมดุลกับ write
- ▶ เรียนรู้ต่อเนื่อง: Query Store, HA, Cloud